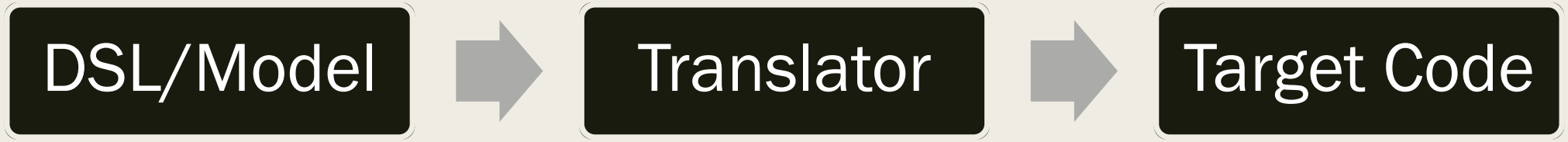
SDU
University of Southern Denmark

PROGRAM ANALYSIS
for domain specific languages

Jyoti Prakash

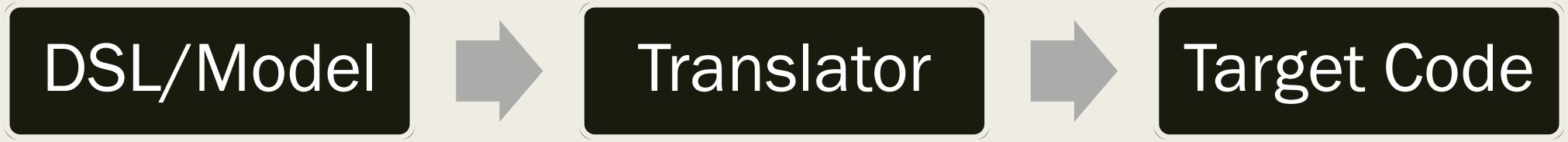
Model Driven Software Development, Spring 2026

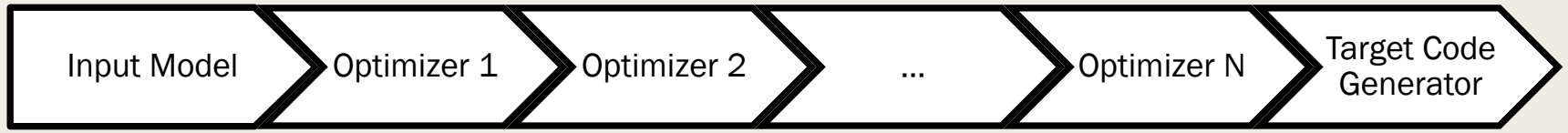
Model Based Software Engineering



- How do you optimise the model before generating the target code?
  - *Traditional compiler optimization techniques help!*

Optimising Models





- How do you optimise the model before generating the target code?
  - *Traditional compiler optimization techniques help!*
- Translators runs a series of optimisers before generating the target code .

Program Analysis?
I have seen it before…

- Advanced Software Engineering Methdologies
  - *Program Slicing*
  - *Taint Analysis*
  - *Semantic Program Repair*
  - *Symbolic Execution*
  - *Data flow analysis and Control Flow Graph*
- This lecture: Deeper dive into data flow analysis techniques
  - *Goal: for optimizing DSLs*
    - Constant Folding
    - Simplify Expressions

Use of dataflow analysis: Bug Detection

Ariane 5

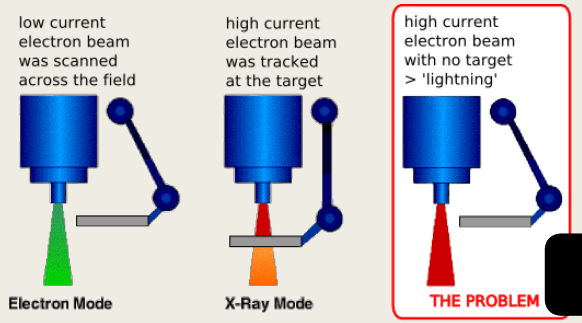
Shell Shock

CrowdStrike

Simulating and Preventing CVE-2021-44228 Apache Log4j RCE Exploits

CVE Advisory

CVE-2024-3094 – Dangerous XZ Utils Backdoor is Discovered



Therac

tray including the target, a flattening filter, the collimator jaws and an ion chamber was moved OUT for "electron" mode, and IN for "photon" mode.

CVE-2022-28756: Zoom Meetings Privilege Escalation Flaw

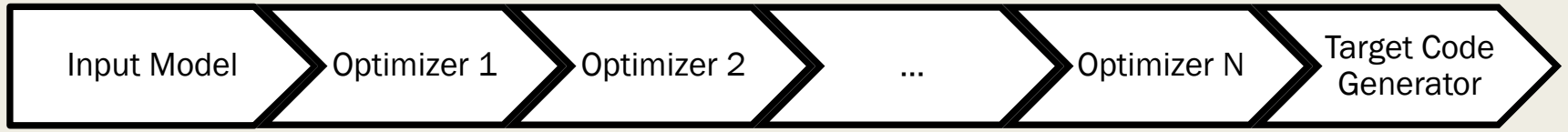
CVE-2022-28756 is a privilege escalation vulnerability in Zoom Meetings for macOS that allows low-privileged users to gain root access. This article covers the technical details, affected versions, impact, and mitigation.

SDU
University of Southern Denmark

Data Flow Analysis in DSL: Practice

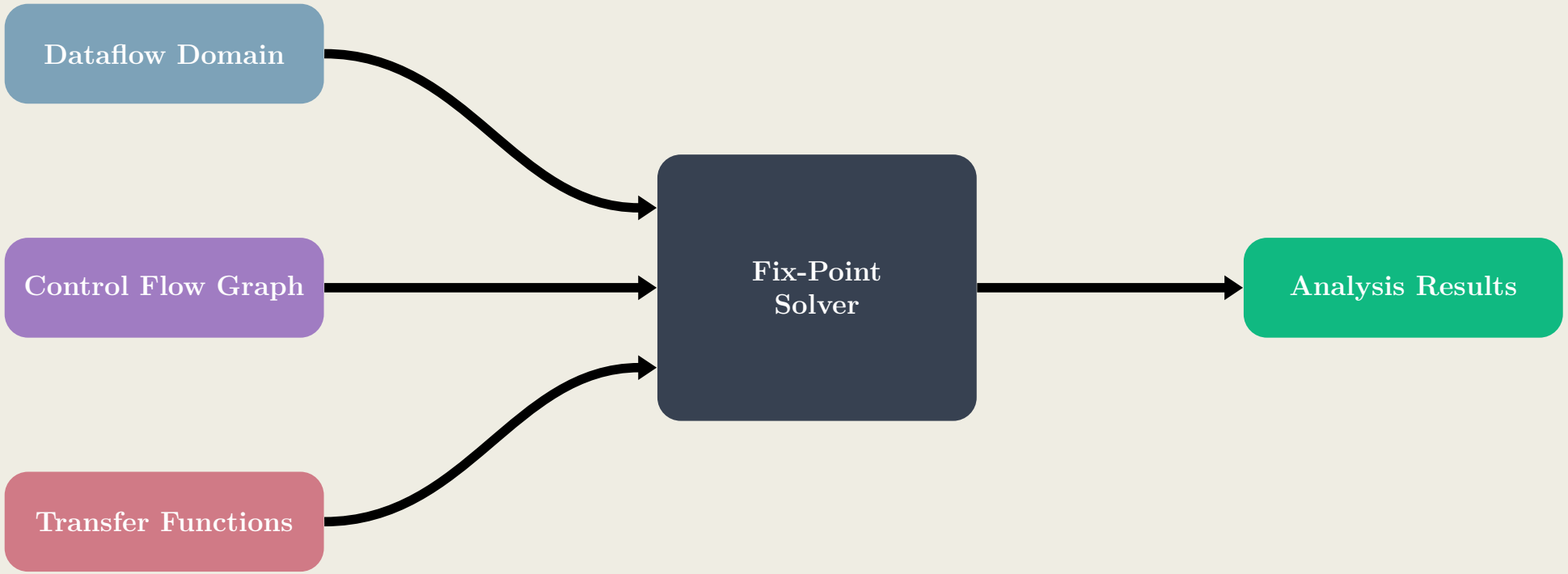
DSL in Practice	What they analyse
Matlab	Matrix operations
Simulink	Optimizing generated code Safety of Simulink models
Verilog	Dead Signals
SQL	Path Optimization
MLIR	Tensor Transformation
Halide	Image processing

Optimisation



- Each optimisation is transforming an unoptimized AST to an optimized AST
  - $optimisation: AST \rightarrow AST$
  - *function that takes an AST an input and returns AST as output*
- One way to implement: write a unique optimisation with the common interface
  - *Redundant effort*
  - *Difficult to maintain; end up with a bunch of optimisers*
- Cleaner way: Data flow analysis

Dataflow Analysis Architecture



Data flow Domain

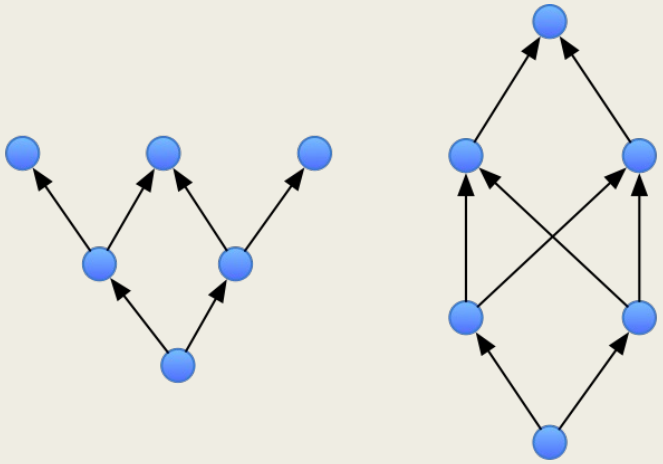
- Data flow analysis typically operates on *abstract values* rather than concrete program values.
  - *The goal is to simplify reasoning while preserving the properties of interest.*
  - *Example: In taint analysis, the exact value of a variable is irrelevant.*
    - The analysis tracks a binary property: whether a variable is **tainted** or **untainted**.
- **Key Question:** “What exact data does a variable hold?” to “What kind of data does the variable hold?”
- *Data flow domain may be infinite but must be represented in finite and computable forms*
  - *Instead of representing integers, a data flow analysis to determine the sign of a numerical variable can only track +, -, unknown, or can’t determined*

Comparing Values in Data flow Domain

- We have an abstract domain $\{tainted, untainted\}$
- How do we algorithmically define if a variable x is tainted in then-branch of if-else statement while not tainted in else-branch
  - *Technique to compare these values*
- **Key Question:** What value should take precedence?
- Technique to compare → Partial Orders
  - *Introduce a relation $\sqsubseteq$ over these values*
  - *Example: $untainted \sqsubseteq tainted$*

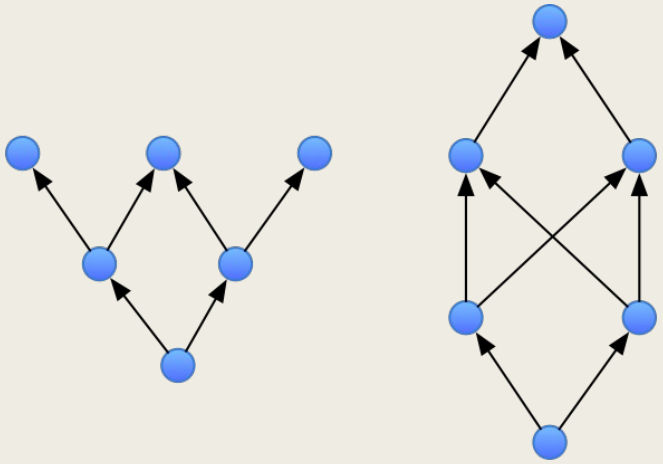
```
if (condition) {
   x = input() // tainted
} else {
   x = 5 // untainted
}
```

Partial Order



- Given a set S, a partial order $\sqsubseteq$ is a binary relation that satisfies
  - *Reflexivity* , $\forall x \in S,\ x \sqsubseteq x$
  - *Anti-symmetric,* $\forall x,y \in S,\ x \sqsubseteq y,\ y \sqsubseteq x \Longrightarrow x = y$
  - *Transitivity,* $\forall x,y,z \in S,\ x \sqsubseteq y,\ y \sqsubseteq z \Longrightarrow x \sqsubseteq z$
- Example: the partial order $(N, \leq)$
  - *set of natural numbers less than k N = {1, 2, …., k}*
  - *Ordered by the relation* $\leq$
- Finite ones can be illustrated with Hasse diagram.

Partial Order



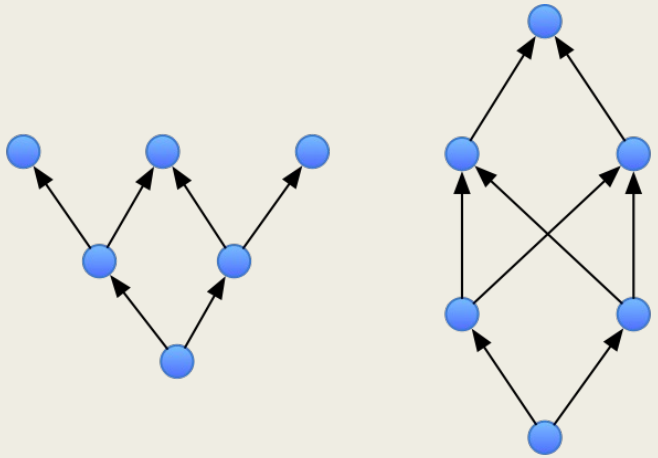
- Given a set S, a partial order $\sqsubseteq$ is a binary relation that satisfies
  - *Reflexivity* , $\forall x \in S,\ x \sqsubseteq x$
  - *Anti-symmetric*, $\forall x,y \in S,\ x \sqsubseteq y,\ y \sqsubseteq x \Longrightarrow x = y$
  - *Transitivity*, $\forall x,y,z \in S,\ x \sqsubseteq y,\ y \sqsubseteq z \Longrightarrow x \sqsubseteq z$
- Example: the partial order $(N_k, \leq)$
  - *set of natural numbers less than k:* $N_k = \{1,2,3,4\}$
  - *Ordered by the relation* $\leq$
- Finite ones can be illustrated with Hasse diagram.

Quiz: Which of these are partial orders?

- Set of natural numbers less than k: $(N_k, <)$
- Set of natural numbers $(N_k, \{ (x, y) \in R, x \mid y \})$ with $x \mid y$ means x is divisible by y

Upper and lower bounds

- Let X be a set
- We say that y ∈ S is an upper bound, when $\forall$x∈X, x ⊑ y
  (it's is the largest elements in the set)
- We say that y ∈ S is a lower bound, when $\forall$x ∈ X, y ⊑ x
  - *It's is the smallest element in the set*
- A least upper bound (LUB) ⨆X is defined by
  - $X \sqsubseteq \bigsqcup X \wedge \forall y \in S : X \sqsubseteq y \implies \bigsqcup X \sqsubseteq y$
- A greatest lower bound (GLB) ⊓X is defined by
  - $\bigsqcap X \sqsubseteq X \wedge \forall y \in S : y \sqsubseteq X \implies y \sqsubseteq \bigsqcap X$

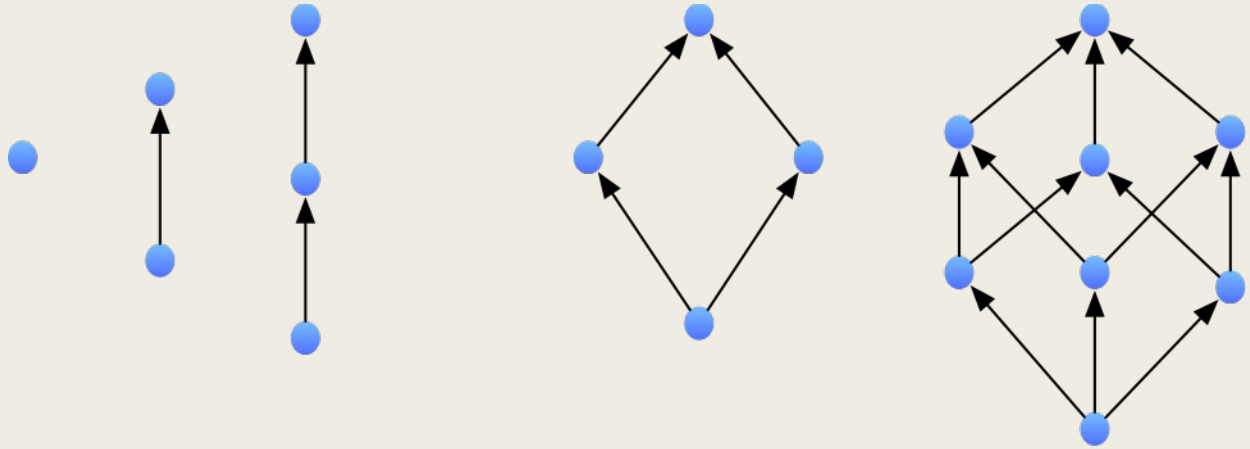


Lattices

- A complete lattice is a partial order
  - *where LUB and GLB exists for all elements in the set*
    $$\sqcup X \text{ and } \sqcap X \text{ exists for all} X \subseteq S$$
- A lattice must have a unique largest and smallest element
  - *Unique upper bound (Top) :* $\top = \sqcup S$
  - *Unique lower bound (Bottom):* $\bot = \sqcap S$
- A finite set S is a lattice if there exists
  - *a least upper bound and a greatest lower bound*
  - *Top and Bottom exists in S*
  - *GLB and LUB exists for all x, y ∈ S*

Lattices

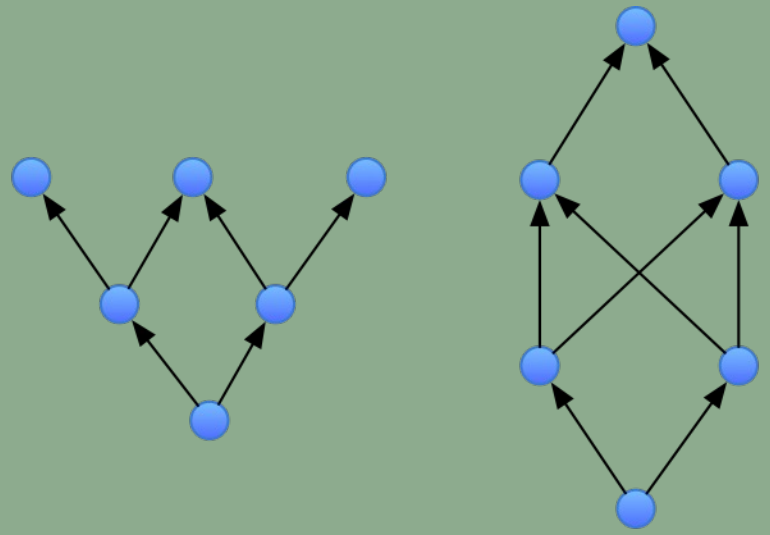
- These partial orders are lattice



SDU
University of Southern Denmark

Quiz

- Are these partial order lattices?



Powerset Lattice

- Every finite set defines a lattice $(2^A, \subseteq)$ where

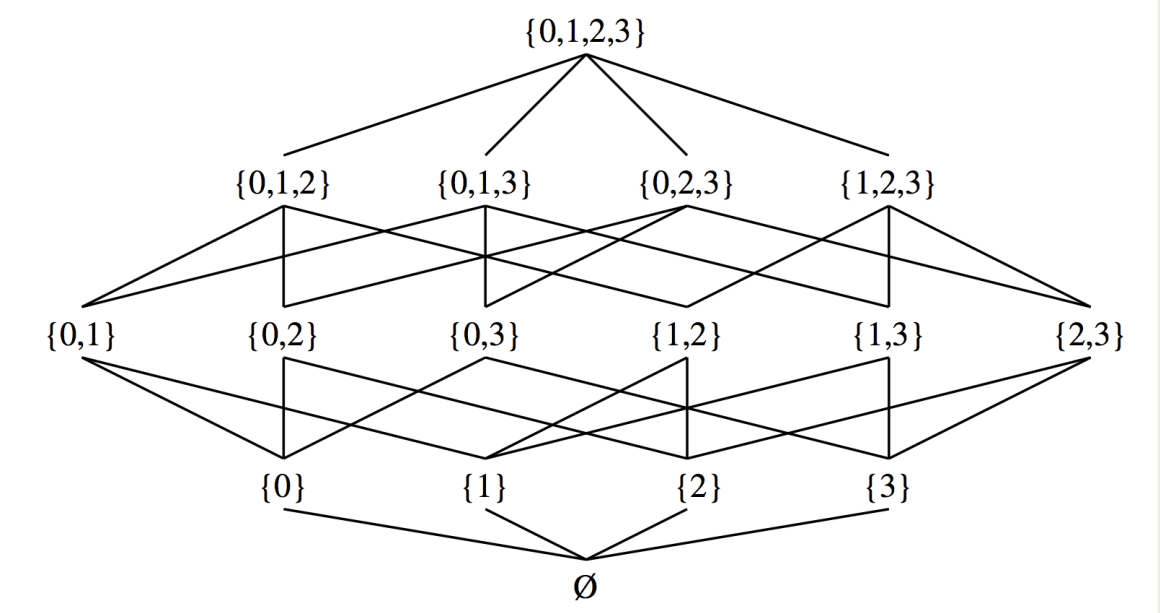
$$\bot = \varnothing$$

$$\top = A$$

$$x \sqcup y = x \cup y$$

$$x \sqcap y = x \cap y$$

- Example: Taint Analysis.
  Domain is a set of tained variables

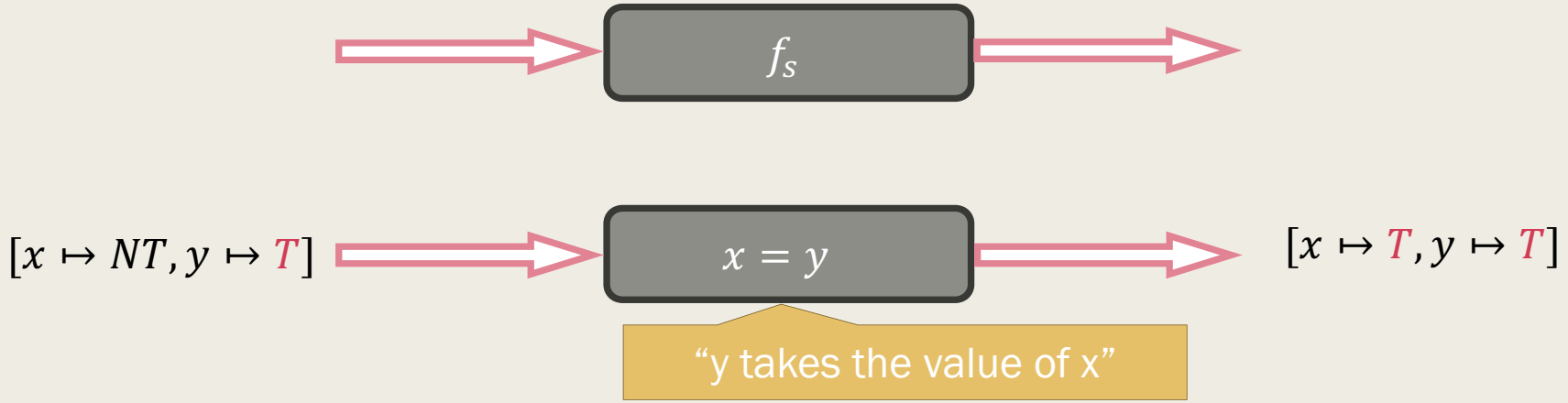


Control Flow Graph

- Data structure to **statically** determine the flow of control in a program
  - *ASTs do not order the control flow*
  - *CFG is computed from the AST*
- Labelled Directed graph on program statements $\mathrm{CFG} = (\mathrm{S}, \mathrm{E})$
  - S *is a set of program statements*
  - E *is a set of edges over statements*
  - *Edges can have an* $empty$, $true$, *or* $false$ *labels.*
- The direction of the edge determines the control flow.
  - $s_1 \rightarrow s_2 \in E$ *if* s_2 *comes after* s_1 *in one of the execution trace*

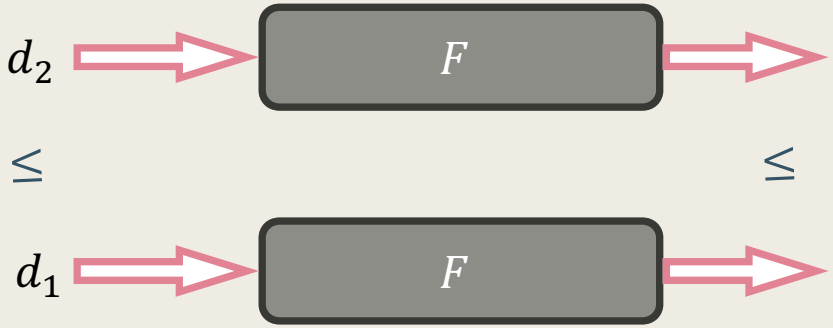
Monotone Dataflow Analysis

- Framework for Dataflow Analysis
  - *CFG: Determines the control flows*
  - *Dataflow Domain: Determines the "thing" that the analysis should track*
  - *Missing piece: How do a statement s modify the domain?*
- Set F of functions $f_s : D \to D$ that maps the statement s for the domain D
  - *Called as transfer functions*



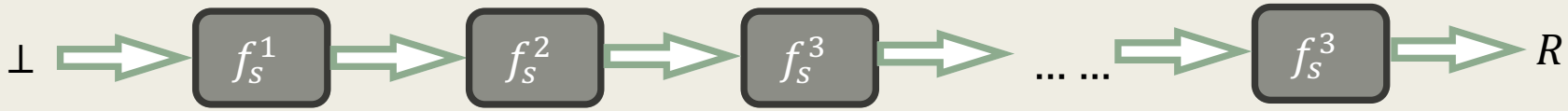
Monotone Dataflow Analysis

- Monotonic Functions
  - $d_1, d_2 \in D$
  - $d_1 \leq d_2 \rightarrow F(d_1) \leq F(d_2)$
- Important Property for transfer functions
  - *Guarantees termination of the analysis when the domain is finite*



Analysis Technique

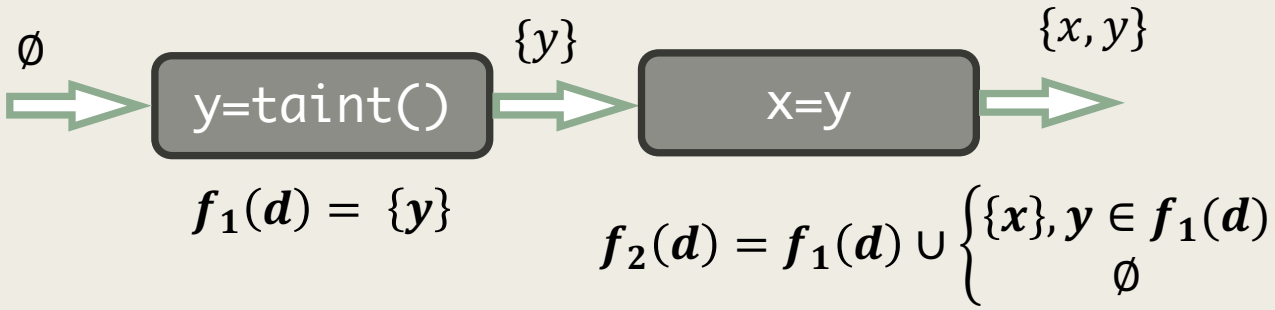
- Chain of transfer functions



- Start with the initial state ⊥ and apply the function f_s^i on the results of the previous statement $R = f_s^n\left(\ldots\left(f_s^3\left(f_s^2\left(f_s^1(\bot)\right)\right)\right)\right)$
- 3 cases for a control flow graphs:
  - *Linear sequence of statements*
    - Straightforward
    - directly apply the mentioned approach
  - *Merging of control flows*
    - Take the join or meet of the incoming edges
  - *Cycles in the CFG*

Program Sequences

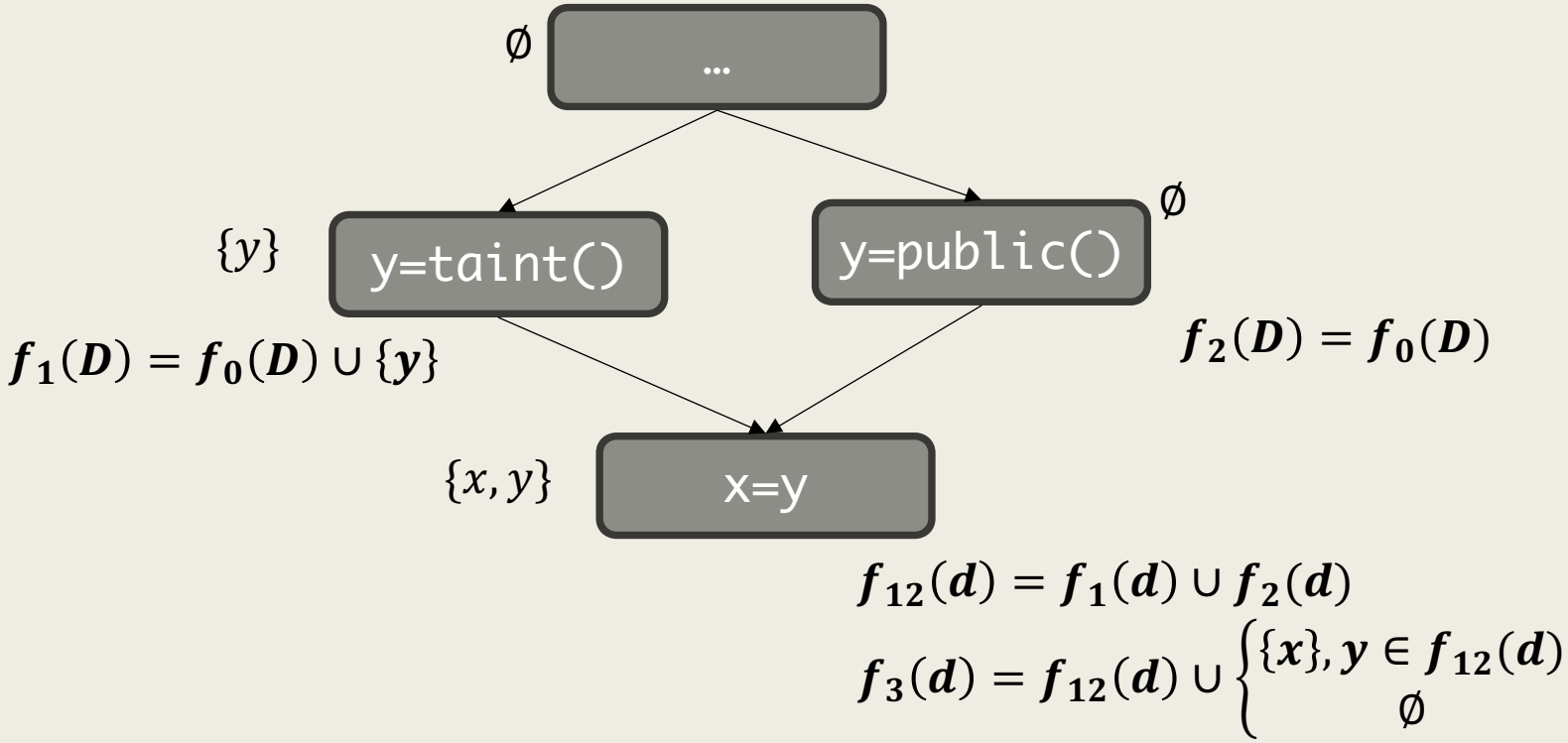
- Taint Analysis: Set of tainted variables



- The result will be direct composition of the two functions $f = f_2 \circ f_1$

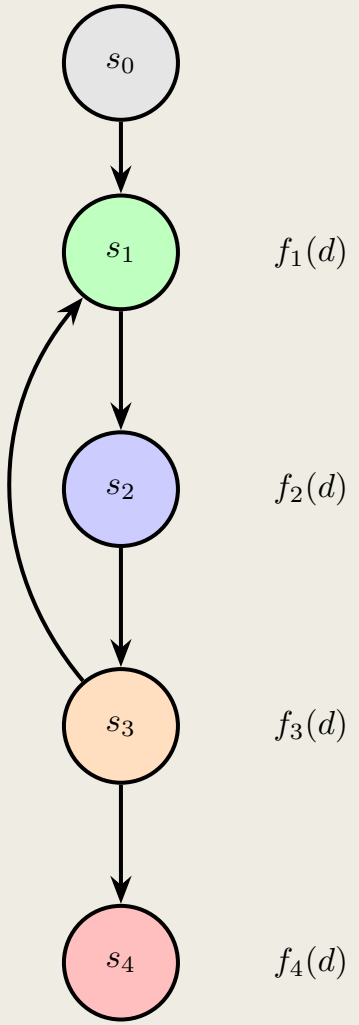
Control Flow Merge

- Taint Analysis: Set of tainted variables



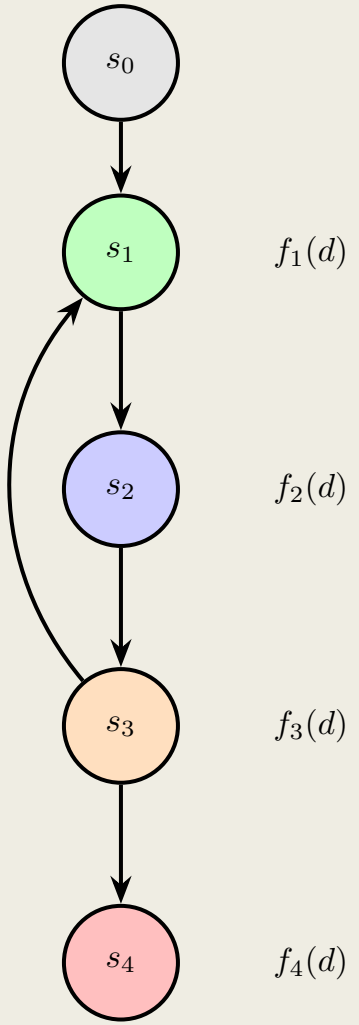
Cycles in CFG

- Cycles in CFG arise due to loops
  - *Merging does not help. Why?*



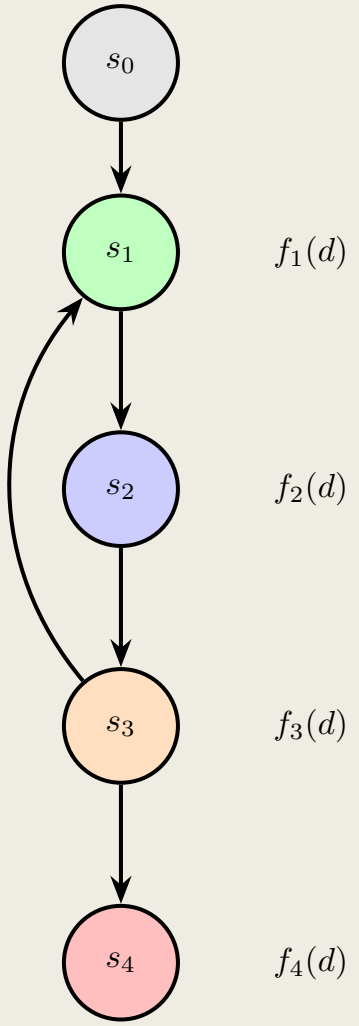
Cycles in CFG

- Cycles in CFG arise due to loops
  - *Merging does not help. Why?*
  - *Merging data flows at s_0 and s_3 at s_1 → calculate s_2 again, …*
  - *May not terminate if we continue to go*
- *Help*: Kanster Tarski Fix Point Theorem
  - *If we have*
    - A finite lattice
    - Monotonic Functions defined over the finite lattice
  - *Recalculating is guarantee to terminate with the value*
- Called as Fixed-Point Iteration



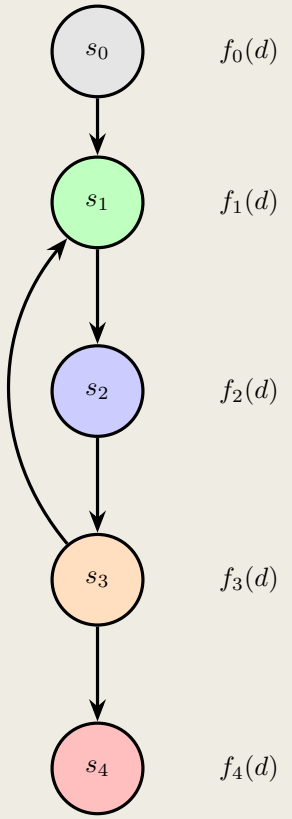
Fixed Point Iteration

- Computes a fix-point values
  - *a stable state when no more changes are possible after repeated application of transfer functions*
- Start with the initial state as input state, say $d_0 = \perp$
- Apply the transfer functions F and compute the analysis on the input state
  - $F = [f_1, f_2, f_3, f_4]$ *is a set of functions defined over an abstract domain* D
  - $d_{i+1} = F(d_i)$
  - *Terminate when* $d_{i+1} = d_i$
- Inefficient technique: It repeatedly computes every function even if it does not change



Fixed Point Iteration – Worklist

- **Observe:**
  - Solution of f_0 *will not change once it has been computed*
  - *Solution of* f_1 *will change if and only if* f_0 *or* f_3 *changes*
- Solution: Worklist Solver
- ***Initialisation***
  - *Initialise the solutions for all functions:* $f_i(d) = \perp$
  - *Add* f_0 *to a queue (called as worklist)*
- ***Iteration***
  - *Pick the first function from the worklist, say* f_i.
  - *Compute the new solution* d_{new}.
- ***Update***
  - *If* $d_{new} \neq d$ *, then add the dependent functions (function for the successors or predecessors' nodes) to the queue from the CFG*
  - *Run until the worklist is empty*



SDU
University of Southern Denmark

What do you need for dataflow analysis?

Dataflow Domain

- Figure out what you are interested in
- Figure out the ordering in that domain

Transfer Functions

- Define the transfer functions on the dataflow domain

Fixed Point Solver

- Use the fixed-point solver that takes the transfer functions and computes the fix-point solution

VecDSL - Our DSL of Choice

- DSL to perform vector operations
  - *Pedagogical tool for teaching program analysis*
  - *NOT something that can be used!*
- Compiles directly to the C++ leveraging the armadillo API
  - *Armadillo is a C++ library for linear algebra*
  - *Performs some level of optimisation for single expressions*
    - c = a+b; d = a+b; // this will not be optimized
- Goal: Define the DSL → Perform optimisations on the DSL → generate the target code

VecDSL Grammar

$$
\begin{aligned}
\mathit{program} &::= \texttt{\{}\ \mathit{stmt}\ \texttt{\}} \\
\mathit{stmt} &::= \texttt{let}\ \mathit{ID}\ \texttt{[=}\ \mathit{expr}\ \texttt{]} \\
&\mid \mathit{ID}\ \texttt{=}\ \mathit{expr} \\
&\mid \texttt{print (}\ \mathit{expr}\ \texttt{)} \\
&\mid \texttt{if (}\ \mathit{expr}\ \mathit{relop}\ \mathit{expr}\ \texttt{)} \\
&\quad\ \texttt{\{}\ \mathit{stmt}\ \texttt{\}} \\
&\quad\ \texttt{[else \{}\ \mathit{stmt}\ \texttt{\}]} \\
&\mid \texttt{while (}\ \mathit{expr}\ \mathit{relop}\ \mathit{expr}\ \texttt{)} \\
&\quad\ \texttt{\{}\ \mathit{stmt}\ \texttt{\}} \\
\mathit{relop} &::= \texttt{==} \mid \texttt{!=} \mid \texttt{<} \mid \texttt{<=} \mid \texttt{>} \mid \texttt{>=} \\
\mathit{expr} &::= \mathit{expr}\ \texttt{(}\ \texttt{+} \mid \texttt{-}\ \texttt{)}\ \mathit{expr} \\
&\mid \mathit{expr}\ \texttt{\#}\ \mathit{expr} \\
&\mid \mathit{expr}\ \texttt{(}\ \texttt{*} \mid \texttt{/}\ \texttt{)}\ \mathit{expr} \\
&\mid \texttt{-}\ \mathit{expr} \\
&\mid \mathit{expr}\ \texttt{->} \\
&\quad\ \texttt{(}\ \texttt{tpos} \mid \texttt{len} \mid \texttt{dim} \mid \mathit{ID} \mid \mathit{NUMBER}\ \texttt{)} \\
&\mid \texttt{[}\ \mathit{expr}\ \texttt{\{ ,}\ \mathit{expr}\ \texttt{\}]} \\
&\mid \mathit{ID} \mid \mathit{NUMBER} \mid \texttt{(}\ \mathit{expr}\ \texttt{)}
\end{aligned}
$$

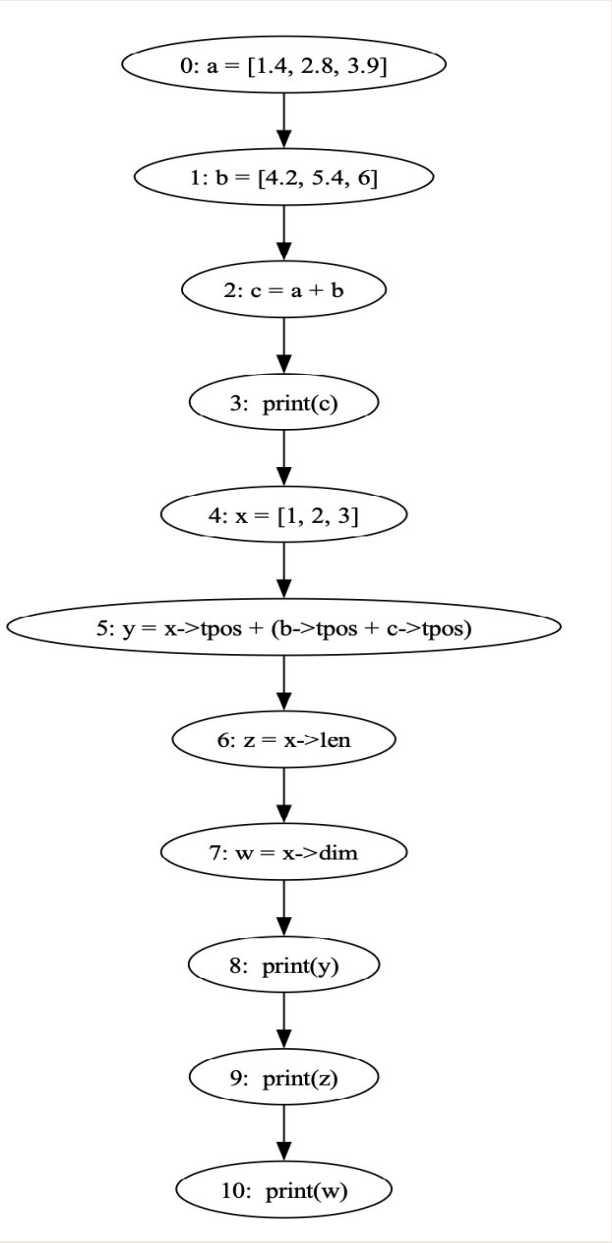
VecDSL Programs

```
-- Vector addition demo
a = [1.4, 2.8, 3.9]
b = [4.2, 5.4, 6]
c = a + b
print(c)

-- Demonstrate transpose, length, and
-- dimension operators
x = [1, 2, 3]
y = x.tpos + (b.tpos + c.tpos)
z = x.len
w = x.dim
print(y)
print(z)
print(w)
```

```
let a = [1, 2, 3]
let b = [4, 5, 6]
let offset = [0, 0, 1]
let sum = b + offset
let loopSeed = a->0
let loopLimit = 3
let first = a->0
let loopValue = [0,0,0]
while (loopSeed <
loopLimit) {
    loopValue = sum + b
    print(loopValue)
    loopSeed = loopSeed +
1
}

if (first > 0) {
    print(sum)
} else {
    print(a)
}
```

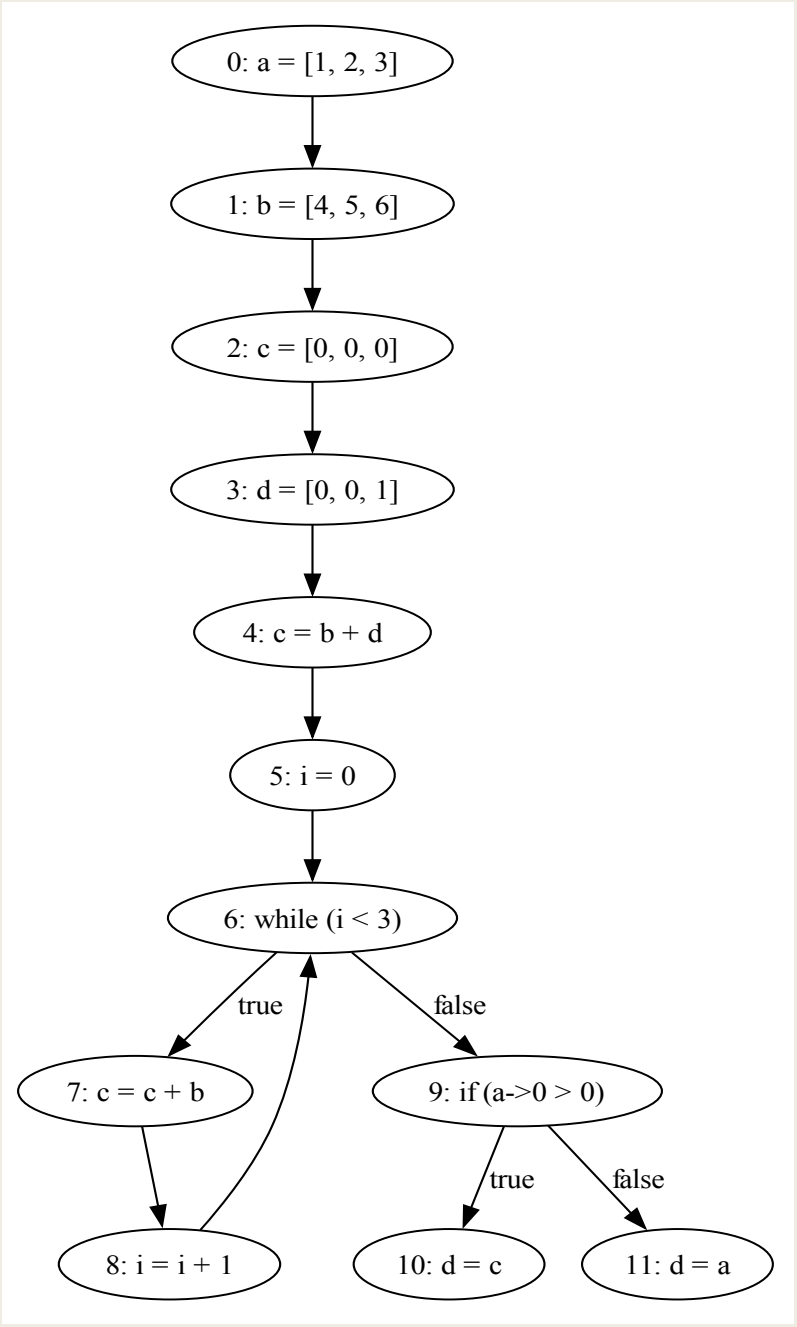
Control Flow Graph

```
-- Vector addition demo
a = [1.4, 2.8, 3.9]
b = [4.2, 5.4, 6]
c = a + b
print(c)

-- Demonstrate transpose, length, and
-- dimension operators
x = [1, 2, 3]
y = x->tpos + (b->tpos + c->tpos)
z = x->len
w = x->dim
print(y)
print(z)
print(w)
```



Control Flow Graph

```
let a = [1, 2, 3]
let b = [4, 5, 6]
let offset = [0, 0, 1]
let sum = b + offset
let loopSeed = a->0
let loopLimit = 3
let first = a->0
let loopValue = [0,0,0]
while (loopSeed <
loopLimit) {
    loopValue = sum + b
    print(loopValue)
    loopSeed = loopSeed +
1
}

if (first > 0) {
    print(sum)
} else {
    print(a)
}
```



Optimizations with VecDSL

- Constant Folding

```
a = 2 + 3 * 4
b = [1 + 2, 3 * 4, 10 - 5]
c = [ (2 + 3) * 4, 6 / 2, 8 - 3 * 2 ]
d = [ [1 + 1, 2 * 2], [3 + 3, 4 * 1] ]
```

```
a = 14
b = [ 3, 12, 5 ]
c = [ 20, 3, 2 ]
d = [ [2, 4], [6, 4] ]
```

- Double Transpose Elimination

```
x = [1, 2, 3]
y = tpos x
z = tpos y
```

```
x = [1, 2, 3]
y = tpos x
z = x
```

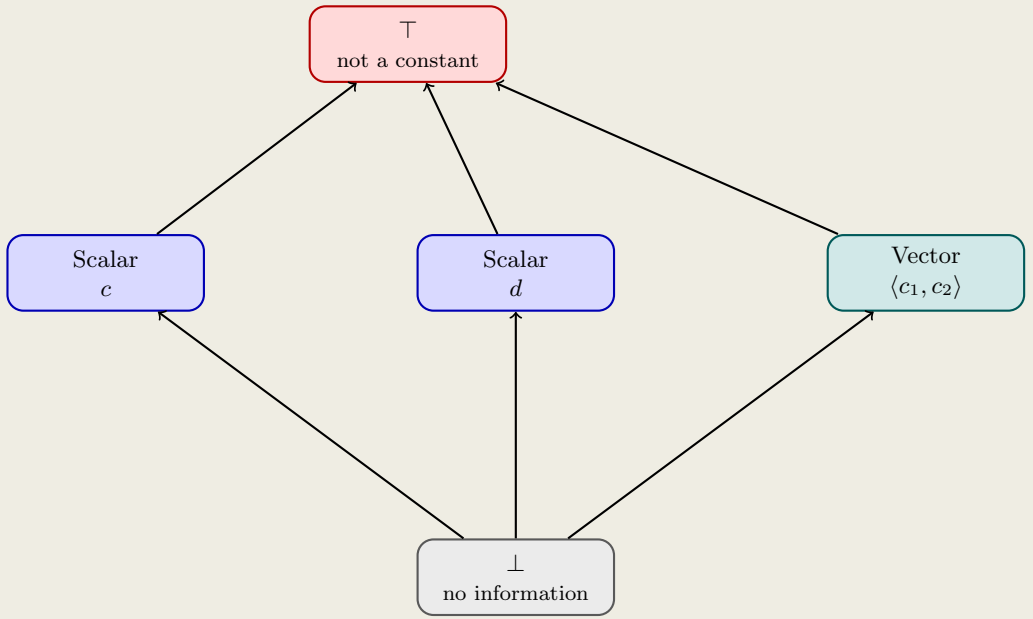
- Factorizing repeated sub-expressions

```
a = [1, 2, 3]
b = (a # a) + (a # a)
c = (a # a) * (a # a)
```

```
a = [1, 2, 3]
tmp = a # a
b = tmp + tmp
c = tmp * tmp
```

Constant Propagation

- Domain
  - *Bottom element: No information*
  - *Scalars and Vectors*
  - *Top of the element: Not a constant*
- Assume that the arithmetic operations are done on compatible types
  - *No scalar + vector addition*
  - *No vector division*
  - *In practice, this will again require a data flow analysis over types.*



Abstract Operators

- Define how the operators (such as +, -, *, /, #) behave in the constant domain
  - $evaluate(expr)$ *performs the evaluation on constant operation*
- Intuition:
  - *Any arithmetic operation with* ⊤ *results in* ⊤
  - *Any arithmetic operation of a scalar or vector with* ⊥ *results in the corresponding scalar or vector*
  - *Rest corresponding to their concrete values*

Operators	Result
$1 \mathbin{\widehat{+}} \top$	⊤
$[1,2,3] \mathbin{\widehat{\#}} \top$	⊤
$\bot \mathbin{\widehat{\#}} [1,2,3]$	$[1,2,3]$

$$
\begin{aligned}
expr ::= \; & expr \;\texttt{(}\; \texttt{+} \mid \texttt{-} \;\texttt{)}\; expr \\
\mid \; & expr \;\texttt{\#}\; expr \\
\mid \; & expr \;\texttt{(}\; \texttt{*} \mid \texttt{/} \;\texttt{)}\; expr \\
\mid \; & \texttt{-}\; expr \\
\mid \; & expr \;\texttt{->} \\
& \quad \texttt{(}\; \texttt{tpos} \mid \texttt{len} \mid \texttt{dim} \mid ID \mid NUMBER \;\texttt{)} \\
\mid \; & \texttt{[}\; expr \;\texttt{\{}\; \texttt{,}\; expr \;\texttt{\}}\; \texttt{]} \\
\mid \; & ID \mid NUMBER \mid \texttt{(}\; expr \;\texttt{)}
\end{aligned}
$$

Transfer Functions

```
stmt ::= let ID [ = expr ]
       | ID = expr
       | print ( expr )
       | if ( expr relop expr )
           { stmt }
           [ else { stmt } ]
       | while ( expr relop expr )
           { stmt }
```

- Track state of the variables for each statement
  - *Idea: Map the variables to the constant lattice (still a lattice)*
    $State: Vars \mapsto ConstLattice$
- Assignment: $ID = expr$
  - $f(s_i) = State[ID \mapsto evaluate(expr)]$
  - *Same goes for declaration (let statements)*
- Control Flow Join (If Else and While)
  - $f(s_i) = \sqcup f(s_j)$ *where* s_j *are the statements that are predecessors of* s_i *in the CFG*
- Print Statement: Identity operations

Constant Propagation

Step	Worklist	Current Node	Added Nodes	State (a, b, c, x, y, z)
0	[1]	–	–	(⊥, ⊥, ⊥, ⊥, ⊥, ⊥)
1	[2]	1	[2]	([1.4,2.8,3.9], ⊥, ⊥, ⊥, ⊥, ⊥)
2	[3]	2	[3]	([1.4,2.8,3.9], [4.2,5.4,6], ⊥, ⊥, ⊥, ⊥)
3	[4]	3	[4]	([1.4,2.8,3.9], [4.2,5.4,6], [5.6,8.2,9.9], ⊥, ⊥, ⊥)
4	[5]	4	[5]	([1.4,2.8,3.9], [4.2,5.4,6], [5.6,8.2,9.9], ⊥, ⊥, ⊥)
5	[6]	5	[6]	([1.4,2.8,3.9], [4.2,5.4,6], [5.6,8.2,9.9], [1,2,3], ⊥, ⊥)

```
-- Vector addition demo
a = [1.4, 2.8, 3.9]
b = [4.2, 5.4, 6]
c = a + b
print(c)

-- Demonstrate transpose, length, and
-- dimension operators
x = [1, 2, 3]
y = x->tpos + (b->tpos + c->tpos)
z = x->len
w = x->dim
print(y)
print(z)
print(w)
```

Constant Propagation Example

Step	Worklist	Current Node	Added Nodes	State (a, b, c, x, y, z)
6	[7]	6	[7]	([1.4,2.8,3.9], [4.2,5.4,6], [5.6,8.2,9.9], [1,2,3], [10.8,15.6,18.9], ⊥)
7	[8]	7	[8]	([1.4,2.8,3.9], [4.2,5.4,6], [5.6,8.2,9.9], [1,2,3], [10.8,15.6,18.9], 3)
8	[9]	8	[9]	([1.4,2.8,3.9], [4.2,5.4,6], [5.6,8.2,9.9], [1,2,3], [10.8,15.6,18.9], 3)
9	[10]	9	[10]	([1.4,2.8,3.9], [4.2,5.4,6], [5.6,8.2,9.9], [1,2,3], [10.8,15.6,18.9], 3)
10	[11]	10	[11]	([1.4,2.8,3.9], [4.2,5.4,6], [5.6,8.2,9.9], [1,2,3], [10.8,15.6,18.9], 3)
11	[]	11	[]	([1.4,2.8,3.9], [4.2,5.4,6], [5.6,8.2,9.9], [1,2,3], [10.8,15.6,18.9], 3)

```
-- Vector addition demo
a = [1.4, 2.8, 3.9]
b = [4.2, 5.4, 6]
c = a + b
print(c)

-- Demonstrate transpose, length, and
-- dimension operators
x = [1, 2, 3]
y = x->tpos + (b->tpos + c->tpos)
z = x->len
w = x->dim
print(y)
print(z)
print(w)
```

Constant Propagation with Loop

Step	Worklist	Current Node	Added Nodes	State (x, y, i)
0	[1]	–	–	(⊥, ⊥, ⊥)
1	[2]	1	[2]	([1,2,3], ⊥, ⊥)
2	[3]	2	[3]	([1,2,3], [0,0,0], ⊥)
3	[4]	3	[4]	([1,2,3], [0,0,0], 0)
4	[5,9]	4	[5,9]	([1,2,3], [0,0,0], 0)
5	[6]	5	[6]	([1,2,3], [1,2,3], 0)
6	[7]	6	[7]	([2,3,4], [1,2,3], 0)
7	[4]	7	[4]	([2,3,4], [1,2,3], 1)
8	[5,9]	4	[5,9]	(T, T, T)
9	[6]	5	[6]	(T, T, T)
10	[7]	6	[7]	(T, T, T)
11	[4]	7	[4]	(T, T, T)
12	[9]	4	[9]	(T, T, T)
13	[]	9	[]	(T, T, T)

```
1. x = [1,2,3]
2. y = [0,0,0]
3. i = 0
4. while (i < 2) {
5.    y = y + x
6.    x = x + [1,1,1]
7.    i = i + 1
8. }
```

Useful Constant Propagation with Loop

- The program has an infinite loop.
- Constant Propagation still terminates (verify!)
- It still gives the optimised program! Impossible with running the program

```
1. x = [1,2,3]
2. y = [0,0,0]
3. while (1 < 2) {
4.    y = y + x
5.    print(y)
6. }
```

```
1. x = [1,2,3]
2. y = [0,0,0]
3. while (1 < 2) {
4.    y = [1,2,3]
5.    print(y)
6. }
```

Double Transpose Elimination

```
let x = [1, 2, 3]
let y = x->tpos
let z = y->tpos
let w = z->tpos
print(w)
```

- Observe: x and z have the same values
  - *Transpose of a transpose of a matrix is the matrix itself*
- Context: Each transpose operation takes linear time in terms of size of the vector
  - *These operations can be optimised to save time*
- Goal: find the program points where the variables have been double transposed
- Before doing this, we will need to normalise the operations:
  - *Nested statements: Break x->tpos->tpos into multiple statements with temporaries*
- Lattice: 3 elements
  - *Bottom* ($\perp$)
  - *Transpose* (TR)
  - *Double Transpose* ($TR2$)

Transfer Functions

- Transfer function akin to constant propagation
- Evaluation Function for transpose operation
  - $evaluate(x \rightarrow tpos, env) = \begin{cases} TR, \ if\ env(x) = \perp \\ TR2, if\ env(x) = TR \\ \perp, if\ env(x) = TR2 \end{cases}$
  - *1st case, tracks a transpose operation is the base variable has not been transposed*
  - *2nd case, lifts it to to TR2 if the base variable is a result of an transposed operation (double transpose)*
  - *3rd case, resets it*

Quiz

- Compute the double transpose analysis for the program

```
let x = [1, 2, 3]
let y = x->tpos
let z = y->tpos
let w = z->tpos
print(w)
```

Optimising Double Transpose

- The analysis gives the program points and the variables
  - *Tracking the variable is still missing*
  - *It needs an analysis to track the variables*
    - Example: The analysis will give (z, 3) which denotes the variable z is double transposed at line 3.
- Tracking is possible with a data structure called as use-def graph
  - *Gives the relation between declaration of a variable and its use*
- Other ideas: **Tracking variables??**

```
let x = [1, 2, 3]
let y = x->tpos
let z = y->tpos
let w = z->tpos
print(w)
```

```
let x = [1, 2, 3]
let y = x->tpos
let z = x
let w = z->tpos
print(w)
```

SDU University of Southern Denmark

Recap: Optimizations with VecDSL

- Constant Propagation

```
a = 2 + 3 * 4
b = [1 + 2, 3 * 4, 10 - 5]
c = [ (2 + 3) * 4, 6 / 2, 8 - 3 * 2 ]
d = [ [1 + 1, 2 * 2], [3 + 3, 4 * 1] ]
```

```
a = 14
b = [ 3, 12, 5 ]
c = [ 20, 3, 2 ]
d = [ [2, 4], [6, 4] ]
```

- Double Transpose Elimination

```
x = [1, 2, 3]
y = tpos x
z = tpos y
```

```
x = [1, 2, 3]
y = tpos x
z = x
```

- Factorising repeated sub-expressions

```
a = [1, 2, 3]
b = (a # a) + (a # a)
c = (a # a) * (a # a)
```

```
a = [1, 2, 3]
tmp = a # a
b = tmp + tmp
c = tmp * tmp
```

- Activity: Design an analysis for Factorising repeated sub-expressions?

Let’s do it together!

- Factorising Sub-Expressions
  - *Domain: ??*
  - *Transfer functions: ??*

```
a = [1, 2, 3]
b = (a # a) + (a # a)
c = (a # a) * (a # a)
```

```
a = [1, 2, 3]
tmp = a # a
b = tmp + tmp
c = tmp * tmp
```

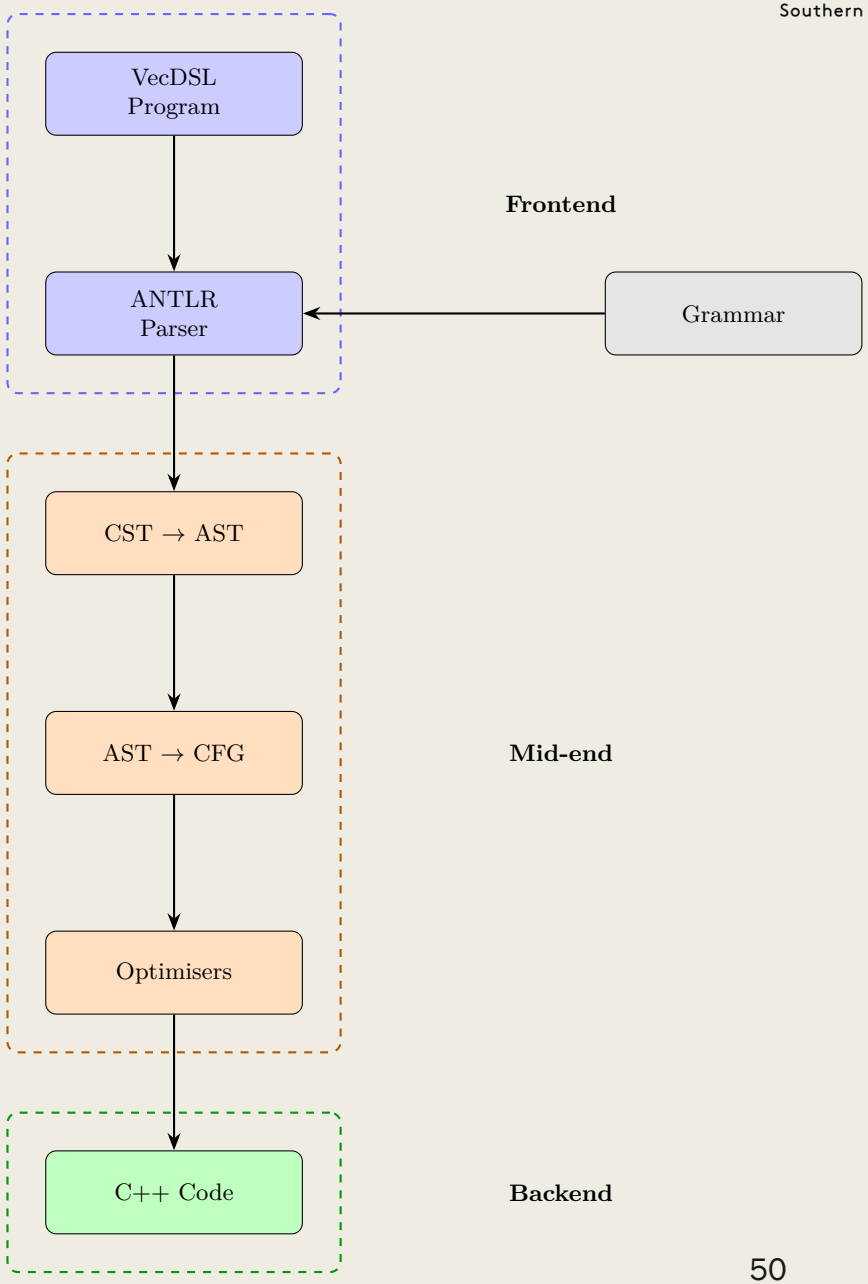
```
stmt ::= let ID [ = expr ]
       | ID = expr
       | print ( expr )
       | if ( expr relop expr )
           { stmt }
           [ else { stmt } ]
       | while ( expr relop expr )
           { stmt }
```

Optimising DSLs

- Often (in declarative DSLs) the problems may not be as complicated
  - *AST rewriting helps → modify the AST*
  - *Rules based transformation of AST*
    - Match the AST and apply the transformations
    - Useful if the language has transformation rules akin to Boolean logic
- Use dataflow analysis if rewriting is not sufficient

VecDSL

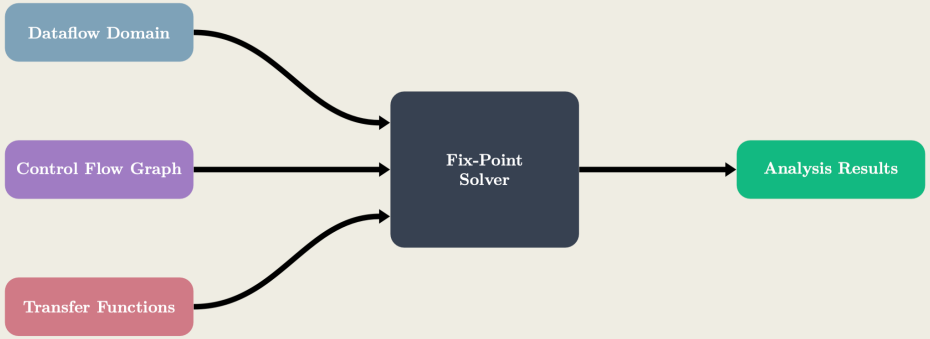
- Compiles to C++ armadillo.
- Implementation at: https://github.com/jpksh90/VecDSL
  - *Not perfect implementation.*
  - *Many parts are generated with GenAI – may contain hallucinations*
  - *Missing implementations:*
    - Type system analysis
    - Array range analysis
  - *Use it as a playground for implementing optimisations*



Program analysis - *for domain specific languages*

SDU University of Southern Denmark

Dataflow Analysis Architecture



Optimizations with VecDSL

- Constant Propagation

```
a = 2 + 3 * 4
b = [1 + 2, 3 * 4, 10 - 5]
c = [ (2 + 3) * 4, 6 / 2, 8 - 3 * 2 ]
d = [ [1 + 1, 2 * 2], [3 + 3, 4 * 1] ]
```

```
a = 14
b = [ 3, 12, 5 ]
c = [ 20, 3, 2 ]
d = [ [2, 4], [6, 4] ]
```

- Double Transpose Elimination

```
x = [1, 2, 3]
y = tpos x
z = tpos y
```

```
x = [1, 2, 3]
y = tpos x
z = x
```

- Factorising repeated sub-expressions

```
a = [1, 2, 3]
b = (a # a) + (a # a)
c = (a # a) * (a # a)
```

```
a = [1, 2, 3]
tmp = a # a
b = tmp + tmp
c = tmp * tmp
```

- Activity: Design an analysis for Factorising repeated sub-expressions?

$$
\begin{aligned}
\mathit{program} &::= \texttt{\{}\ \mathit{stmt}\ \texttt{\}} \\
\mathit{stmt} &::= \texttt{let}\ \mathit{ID}\ \texttt{[=}\ \mathit{expr}\ \texttt{]} \\
&\mid \mathit{ID}\ \texttt{=}\ \mathit{expr} \\
&\mid \texttt{print (}\ \mathit{expr}\ \texttt{)} \\
&\mid \texttt{if (}\ \mathit{expr}\ \mathit{relop}\ \mathit{expr}\ \texttt{)} \\
&\quad \texttt{\{}\ \mathit{stmt}\ \texttt{\}} \\
&\quad \texttt{[else \{}\ \mathit{stmt}\ \texttt{\}]} \\
&\mid \texttt{while (}\ \mathit{expr}\ \mathit{relop}\ \mathit{expr}\ \texttt{)} \\
&\quad \texttt{\{}\ \mathit{stmt}\ \texttt{\}} \\
p &::= \texttt{== |!= |< |<= |> |>=} \\
r &::= \mathit{expr}\ \texttt{(}\ \texttt{+ |-}\ \texttt{)}\ \mathit{expr} \\
&\mid \mathit{expr}\ \texttt{\#}\ \mathit{expr} \\
&\mid \mathit{expr}\ \texttt{(}\ \texttt{* |/}\ \texttt{)}\ \mathit{expr} \\
&\mid \texttt{-}\ \mathit{expr} \\
&\mid \mathit{expr}\ \texttt{->} \\
&\quad \texttt{(}\ \texttt{tpos |len |dim |}\mathit{ID}\ \texttt{|}\mathit{NUMBER}\ \texttt{)} \\
&\mid \texttt{[}\ \mathit{expr}\ \texttt{\{ ,}\ \mathit{expr}\ \texttt{\}]} \\
&\mid \mathit{ID}\ \texttt{|}\mathit{NUMBER}\ \texttt{|(}\ \mathit{expr}\ \texttt{)}
\end{aligned}
$$

References

- Slides for lattice taken from Møller and Schwartzbach lecture notes on Static Program Analysis
  - *https://cs.au.dk/~amoeller/spa/spa.pdf*
- Principles of Program Analysis – Neilson, Nielson, and Hankin
  - *https://link.springer.com/book/10.1007/978-3-662-03811-6*
- Static Analyses of Interlanguage Interoperations (Section 2.3) – Jyoti Prakash
  - *https://jpksh90.github.io/assests/thesis.pdf*